Detecting and Repairing Hallucinations in LLM-Generated Answers

Course: Natural Language Processing **Project type:** Team project **Document:** Problem statement, proposed solution, goals, and implementation plan

---

Table of Contents

1. Problem Statement
2. Motivation
3. Background and Related Work
4. Project Goals and Research Questions
5. Proposed Solution
6. Datasets and Resources
7. Implementation Plan (Phase-wise)
8. Evaluation Methodology
9. Baselines and Ablations
10. Timeline
11. Team Roles
12. Tech Stack and Tooling
13. Risks and Mitigations
14. Expected Deliverables
15. Limitations and Future Work
16. References and Reference Repositories

---

1. Problem Statement

Large Language Models (LLMs) can produce answers that sound fluent, confident and well-structured, yet contain **incorrect or unsupported information**. These errors are called *hallucinations*. Because the surrounding text is usually correct and well written, a hallucinated claim is hard for a reader to spot, and it can mislead users in settings such as education, research, healthcare and customer support.

Most current approaches either try to **prevent** hallucinations during generation, or **detect** them at the level of a whole answer. Both leave a practical gap: a user receives an answer in which

some statements are wrong and *most* are right, and has no way to tell which is which, and no corrected version.

Our project addresses the following problem:

> Given a question and an LLM-generated answer, (1) identify which individual claims in the answer are hallucinated or unsupported by checking them against available evidence, (2) localize the unreliable parts of the answer, and (3) produce a repaired answer that fixes or removes the unreliable claims using verified information, while **preserving the parts of the answer that were already correct**.

Formal task definition

- **Input:** a question `q` and an LLM-generated answer `a`.
- **Intermediate output:** a set of atomic claims `C = {c1, ..., cn}` extracted from `a`, each with a label `y_i` in {Supported, Contradicted, Unverifiable} and a set of evidence passages `E_i`.
- **Final output:** a repaired answer `a'` such that:
  - every claim in `a'` is supported by evidence (or is explicitly hedged),
  - claims that were already supported in `a` are retained,
  - fluency and relevance to `q` are maintained.

---

2. Motivation

- **Trust:** Users cannot rely on LLM output without verifying it manually, which defeats much of the productivity benefit.
- **Granularity gap:** Answer-level "true or false" detection is uninformative for long answers, because a single wrong fact can sit inside an otherwise correct paragraph. Prior work finds that claim-level checking and editing gives more precise correction and better preservation of correct content.
- **Detection without repair is incomplete:** Flagging an error leaves the user to fix it. Deleting the error leaves an incomplete answer. A useful system should *repair*.
- **Inspiration from adjacent work:** A recent paper on hallucinations in LLM-generated *code* (Khati et al., FORGE '26) argues for a pipeline that is deterministic and verifiable: parse the output into structural units, validate each against a ground-truth knowledge source, and apply localized edits. It also evaluates *detection* and *fix accuracy* separately. We adapt this design philosophy, **decompose, verify against ground truth, edit locally**, from code to natural-language answers.

---

3. Background and Related Work

3.1 Types of hallucination

- **Factuality hallucination:** the content conflicts with real-world facts (for example, a wrong date or an invented entity).
- **Faithfulness hallucination:** the content conflicts with a provided source or context (for example, a summary that adds facts not in the document).
- **Fine-grained types** (used by FAVA and PFME): entity errors, relation errors, contradictory statements, invented content, subjective statements, and unverifiable statements.

This project focuses primarily on **factuality hallucinations in open-domain question answering**, and treats "unverifiable" claims as a separate category.

3.2 Existing approaches

Approach	**Idea**	**Limitation**
SelfCheckGPT	Sample several answers and measure consistency, with no external evidence	Consistent answers can still be wrong
FActScore	Split text into atomic facts and compute the share supported by a knowledge source	Metric only; does not repair
FacTool	Claim-level verification using tools and search	The user must supply the claims
RARR	Generate verification questions, retrieve evidence, and edit the answer	Edits the whole response, which can drift from the original
Chain-of-Verification (CoVe)	The LLM generates and answers its own verification questions, then revises	Depends on the same model that hallucinated
RefChecker	Represents claims as (subject, predicate, object) triplets	Primarily detection

Approach	Idea	Limitation
FAVA / PFME	Fine-grained error typing with span editing and retrieval	Heavier setup, depends on the retrieval quality
Constrained decoding (PICARD, Synchromesh)	Restrict generation to a grammar	Addresses syntax, not facts

3.3 Gap we target

We combine the strongest ideas into one controlled, **claim-level, evidence-grounded, minimal-edit** pipeline with a **re-verification loop**, and we evaluate it on both **detection quality** and **repair quality** (including how much correct content is preserved).

---

4. Project Goals and Research Questions

4.1 Primary goals

1. **G1: Detect** hallucinated claims in LLM answers by verifying each atomic claim against retrieved evidence.
2. **G2: Localize** the unreliable parts of an answer at the span or claim level, so a reader can see exactly what is doubtful.
3. **G3: Repair** the answer using verified evidence, changing only the unreliable claims.
4. **G4: Evaluate** rigorously, reporting detection metrics, repair metrics, preservation of correct content, and human judgments.

4.2 Secondary goals

- **G5:** Compare design choices (claim granularity, retriever type, verifier type) through ablations.
- **G6:** Produce a small interactive demo that highlights flagged claims and shows original and repaired answers side by side.
- **G7:** Analyze failure modes systematically.

4.3 Research questions

- **RQ1:** How accurately can an evidence-grounded, claim-level pipeline detect hallucinated claims compared with answer-level and no-evidence baselines?
- **RQ2:** Can flagged claims be repaired automatically using retrieved evidence, and how often is the repair correct?
- **RQ3:** Does claim-level editing preserve correct content better than rewriting the whole answer?

- **RQ4:** How do retrieval quality and verifier choice affect end-to-end performance?

4.4 Success criteria

These are targets, not guarantees; we will report whatever we actually measure.

- The pipeline runs end to end on at least 200 test answers.
- It outperforms at least two baselines on claim-level detection F1.
- Repair increases the FActScore-style supported-claim fraction compared with the unrepaired answers.
- It preserves correct claims at a high rate, measured explicitly.
- A documented error analysis explains the main failure categories.

---

5. Proposed Solution

5.1 Overview

We propose a **modular five-stage pipeline**:

Question q + LLM answer a

|

v

[Stage 1] Claim Decomposition -> atomic, self-contained claims

|

v

[Stage 2] Evidence Retrieval -> top-k passages per claim

|

v

[Stage 3] Claim Verification -> Supported / Contradicted / Unverifiable

|

v

[Stage 4] Localization and Repair -> minimal edits to flagged claims only

|

v

[Stage 5] Re-verification -> check the repaired answer, iterate if needed

|

v

Final repaired answer a' (+ report of what changed and why)

5.2 Stage 1: Claim decomposition

Purpose: Break the answer into the smallest independently checkable statements.

Details:

- Split the answer into sentences, then into atomic claims.
- **Decontextualize** each claim so that it stands alone (for example, replace "He was born in 1950" with "Albert Example was born in 1950").
- Keep a **mapping from each claim back to its source span** in the original answer. This mapping is what allows localized repair later.

Candidate methods:

- Few-shot LLM prompting that outputs a JSON list of claims with their source sentence IDs.
- (Optional) a fine-tuned seq2seq model.

Quality checks:

- Manually review a sample of about 50 answers for atomicity (one fact per claim), completeness (no fact lost), and decontextualization.

5.3 Stage 2: Evidence retrieval

Purpose: Find passages that can confirm or refute each claim.

Details:

- **Corpus:** Wikipedia (dump or HuggingFace dataset), chunked into passages of about 100 to 200 words with the page title prepended.
- **Retrievers to compare:**
    1. **BM25** (sparse, strong baseline)
    2. **Dense retrieval** (a sentence-embedding model plus a FAISS index)
    3. **Hybrid** (BM25 plus dense, with a reranker if time permits)
- **Query formulation:** use the decontextualized claim itself, and optionally a reformulated question form of it.
- **Output:** top-k passages (k between 3 and 10) per claim.
- **Optional extension:** live web search API as a second evidence source.

Quality check: Evaluate retrieval on FEVER (is the gold evidence in the top-k?) before building on top of it.

5.4 Stage 3: Claim verification (detection)

Purpose: Decide whether each claim is supported by its evidence.

Labels:

- **Supported:** evidence entails the claim.
- **Contradicted:** evidence conflicts with the claim.
- **Unverifiable:** evidence is insufficient (not enough info).

Verifier options (compared experimentally):

1. **NLI model:** an off-the-shelf DeBERTa-style model fine-tuned on MNLI, FEVER, and ANLI. The claim is the hypothesis and the evidence is the premise.
2. **LLM-as-judge:** a prompt containing the claim and the evidence, asking for a label and a brief justification.
3. **Fine-tuned verifier:** fine-tune a small encoder model on FEVER.

Aggregation:

- Combine evidence from multiple passages (for example, a claim is supported if any passage supports it and none contradicts it).
- Compute a claim-level confidence score.
- Roll up to an answer-level reliability score (the fraction of supported claims).

Output: a table of `(claim, label, confidence, evidence, source span)`.

5.5 Stage 4: Localization and repair

Purpose: Produce a corrected answer that changes as little as possible.

Localization: Using the claim-to-span mapping, highlight the exact spans of the original answer that correspond to Contradicted or Unverifiable claims.

Repair policy by label:

Claim label	**Action**
Supported	**Keep unchanged**
Contradicted, with evidence giving the correct fact	**Rewrite** the claim using the evidence
Contradicted, with no clear replacement	**Remove** the claim or **hedge** it
Unverifiable	**Hedge** (for example, "reportedly") or remove, depending on a configurable setting

Repair method:

- Provide an LLM with: the original sentence, the flagged claim, the verified evidence passages, and a **strict instruction** to use only that evidence, edit only the flagged span, and keep the rest of the sentence intact.
- Constrain the output format (for example, return only the edited sentence) to prevent unwanted rewriting.
- Apply edits sentence by sentence, then reassemble the answer in the original order.

Design principle (borrowed from the code-hallucination paper): edits are **localized and minimal**, so the correct parts remain untouched and the changes are easy to audit.

5.6 Stage 5: Re-verification

Purpose: Ensure the repair did not introduce new errors.

- Re-run Stages 1 to 3 on the repaired answer.
- If any claim is still unsupported, either run one more repair iteration (capped at 2 iterations) or fall back to removing the claim.
- Record the number of iterations required and whether the answer converged.

5.7 Output report

For transparency, the system returns:

- the original answer with flagged spans highlighted,
- a per-claim verdict table with evidence,
- the repaired answer, with a diff against the original,
- an overall reliability score before and after repair.

---

6. Datasets and Resources

6.1 Detection datasets (answers labeled hallucinated or not)

Dataset	Description	Use in project
HaluEval (RUCAIBox/HaluEval; HuggingFace `pminervini/HaluEval`)	About 35k hallucinated and normal samples across QA, dialogue, summarization and general queries	Main detection data (QA subset)
FELM (hkust-nlp/felm)	About 3.9k responses with segment-level factuality annotations	Fine-grained detection evaluation
WikiBio GPT-3 hallucination (SelfCheckGPT release)	Sentence-level labels on generated biographies	Small, quick benchmark
Factcheck-Bench / Factcheck-GPT	Responses decomposed into claims with annotations	Claim-level evaluation and repair analysis
TruthfulQA	817 questions with correct and plausible-but-wrong answers	Misconception-focused testing
FaithBench (optional)	Human-annotated summarization hallucinations	Only if we extend to summarization

6.2 Evidence and verification resources

Resource	Use
Wikipedia (dump or HuggingFace dataset)	Retrieval corpus
FEVER (about 145k claims labeled Supported, Refuted, Not Enough Info)	Train and evaluate the verifier and retriever
SciFact / HoVer (optional)	Scientific and multi-hop claims
Web search API (optional)	Live evidence beyond Wikipedia

6.3 Our own evaluation set (for repair)

Public datasets mostly label *detection*, not *repair*. To evaluate repair we will build a **small gold set of 100 to 200 answers**:

1. Take questions from HaluEval-QA, TruthfulQA, or similar.
2. Generate answers with an LLM (some will contain real hallucinations).
3. Have team members **annotate each claim** (Supported, Contradicted, or Unverifiable) and write or approve a **gold repaired answer**.
4. Measure inter-annotator agreement (for example, Cohen's kappa) on a subset.

6.4 Data splits

- **Train and dev:** used for tuning prompts, thresholds, and verifier fine-tuning.
- **Test:** held out and used **once** for final reporting.

---

7. Implementation Plan (Phase-wise)

Phase 0: Scope, setup and design decisions (Week 1)

Tasks

- Finalize scope: open-domain factual QA, Wikipedia as primary evidence.
- Write precise definitions of: atomic claim, Supported, Contradicted, Unverifiable, and what counts as a correct repair.
- Create the GitHub repository with the following structure:

  project/

```
├── data/           # raw and processed data (not committed if large)
├── src/
│   ├── decompose.py
│   ├── retrieve.py
│   ├── verify.py
│   ├── repair.py
│   ├── pipeline.py
│   └── evaluate.py
├── notebooks/
├── configs/
├── results/
├── demo/
└── README.md
```

- Set up the environment (Python, `requirements.txt`), API keys, and compute (Colab or Kaggle GPU, or institute GPUs).
- Set up experiment logging (a spreadsheet or Weights & Biases).

Deliverable: repo, written task definitions, working environment.

Phase 1: Literature review and baseline reproduction (Weeks 1 to 2)

Tasks

- Read: RARR, CoVe, FActScore, SelfCheckGPT, RefChecker, FacTool, FAVA or PFME, and the attached code-hallucination paper.
- Run one existing open-source tool end to end (for example RefChecker or FActScore) on a few examples to understand input and output formats.
- Write a related-work summary (this becomes the report's related-work section).

Deliverable: 1 to 2 page literature summary, working baseline run.

Phase 2: Data preparation (Week 2)

Tasks

- Download and inspect HaluEval-QA, FELM, FEVER, TruthfulQA.
- Convert everything to a **unified schema**:

```
{
  "id": "...",
  "question": "...",
  "answer": "...",
  "claims": [{"text": "...", "label": "...", "span": [start, end]}],
  "evidence": ["..."],
  "source_dataset": "..."
}
```

- Build the Wikipedia passage index (chunk, embed, store in FAISS, plus a BM25 index).
- Create train, dev and test splits.
- Begin annotating the repair gold set (Phase 7 depends on this, so start early).

Deliverable: cleaned datasets, retrieval index, splits.

Phase 3: Claim decomposition module (Week 3)

Tasks

- Design and iterate on the decomposition prompt (few-shot, JSON output, decontextualization instructions).
- Add span-mapping logic (claim to source sentence in the original answer).
- Validate on about 50 answers: check atomicity, completeness, and decontextualization.
- Handle edge cases: opinions, questions, hedged statements, and non-factual sentences (these should be skipped rather than verified).

Deliverable: `decompose.py`, a decomposition quality report.

Phase 4: Evidence retrieval module (Weeks 3 to 4)

Tasks

- Implement BM25, dense and hybrid retrieval.
- Evaluate retrieval on FEVER (recall@k) and choose the default configuration.
- Tune chunk size, k, and query formulation.
- Add caching so repeated queries are free.

Deliverable: `retrieve.py`, retrieval comparison table.

Phase 5: Claim verification module (Weeks 4 to 5)

Tasks

- Implement three verifiers: off-the-shelf NLI, LLM-as-judge, fine-tuned FEVER model.
- Evaluate each on FEVER dev and on our annotated claims.
- Implement multi-passage aggregation and confidence scores.
- Calibrate thresholds on dev data (for example, the NLI entailment threshold).
- Measure claim-level and answer-level detection metrics on HaluEval or FELM.

Deliverable: `verify.py`, detection results across verifier options.

Phase 6: Localization and repair module (Weeks 5 to 6)

Tasks

- Implement the repair policy (keep, rewrite, hedge, remove).
- Design the repair prompt with strict constraints (evidence-only, edit-only-flagged-span).
- Implement span replacement and answer reassembly.
- Implement the re-verification loop with an iteration cap.
- Run on dev data and manually inspect at least 30 repairs. Identify recurring issues such as over-editing, evidence copied verbatim, or lost fluency, and refine the prompt.

Deliverable: `repair.py`, `pipeline.py` (end-to-end), sample outputs.

Phase 7: Full evaluation, baselines and ablations (Weeks 6 to 7)

Tasks

- Run the full pipeline on the held-out test set.
- Run all baselines (Section 9).
- Run ablations (Section 9).

- Conduct the human evaluation (Section 8.3).
- Perform error analysis: categorize failures and count each category.

Deliverable: results tables and figures, error analysis.

Phase 8: Demo, report and presentation (Weeks 7 to 8)

Tasks

- Build the demo (Gradio or Streamlit): input a question and answer, view highlighted claims, verdicts, evidence, and the repaired answer.
- Write the final report: introduction, related work, method, experiments, results, error analysis, limitations, conclusion.
- Prepare slides and a short demo video or live demo.
- Clean the repo and write a README with reproduction steps.

Deliverable: report, slides, demo, clean repository.

---

8. Evaluation Methodology

We evaluate **detection** and **repair** separately, mirroring the separation of detection accuracy and fix accuracy used in the code-hallucination paper.

8.1 Detection metrics

- **Claim level:** precision, recall, F1 for the "hallucinated" class (Contradicted or Unverifiable vs. Supported). Because recall without precision hides over-flagging, we report both.
- **Answer level:** accuracy, F1, and AUROC for answer-level hallucination detection.
- **Confusion matrix** for the three-way label (Supported, Contradicted, Unverifiable).
- **Retrieval quality:** recall@k on FEVER.

8.2 Repair metrics

Metric	What it measures	How
Supported-claim rate (FActScore-style)	Share of claims in the answer that are supported	Re-run decomposition and verification on the output, before and after repair

Metric	What it measures	How
Fix rate	Share of hallucinated claims that were correctly repaired	Compare against the gold repaired claims (human judgment)
Preservation rate	Share of originally supported claims still present after repair	Claim-level matching between original and repaired answers
Over-edit rate	Share of supported claims that were wrongly changed or removed	Same matching, inverse
Edit distance / ROUGE / BERTScore vs. original	How far the repaired text drifted from the original	Automatic text similarity
Convergence rate	Share of answers that pass re-verification within the iteration cap	Pipeline logs

8.3 Human evaluation

- Sample 50 to 100 test answers.
- Two annotators rate each repaired answer on: **factual correctness**, **preservation of correct content**, **fluency**, and **minimality of edits** (a 1 to 5 scale).
- Report mean scores and inter-annotator agreement.

8.4 Statistical rigor

- Report results with confidence intervals (bootstrap) where possible.
- Fix random seeds and document all prompts, model versions, and thresholds.

---

9. Baselines and Ablations

9.1 Baselines

Baseline	Description
B0: No repair	Original answer (lower bound)

Baseline	Description
B1: Answer-level detection only	Single true/false judgment on the whole answer
B2: SelfCheckGPT-style consistency	No external evidence
B3: LLM self-repair, no evidence	Ask the LLM to "fix any errors" with no retrieval
B4: CoVe-style	Self-generated verification questions, no external corpus
B5: RARR-style whole-answer rewrite	Evidence retrieval, then rewrite the entire answer
B6: Delete-only	Remove all flagged claims, with no rewriting (analogous to deletion-based repair in prior work)

9.2 Ablations (our pipeline)

Ablation	Variants
Claim granularity	Whole answer vs. sentence vs. atomic claim
Retriever	BM25 vs. dense vs. hybrid
Top-k evidence	k = 1, 3, 5, 10
Verifier	NLI vs. LLM judge vs. fine-tuned
Repair policy	Rewrite-only vs. rewrite+hedge vs. rewrite+remove
Re-verification loop	Off vs. 1 iteration vs. 2 iterations

---

10. Timeline

Week	Phase	Key milestone
1	Phase 0, start of Phase 1	Repo, definitions, environment ready
2	Phase 1 and 2	Literature summary, data and index ready
3	Phase 3, start of Phase 4	Decomposition working and validated
4	Phase 4 and 5	Retrieval chosen, first verifier results
5	Phase 5 and 6	Detection results, first repairs
6	Phase 6 and 7	End-to-end pipeline, baselines running
7	Phase 7 and 8	Ablations, human evaluation, error analysis
8	Phase 8	Report, demo, presentation

Adjust to the real course deadline. Build a **minimal end-to-end pipeline by the end of Week 3 or 4** and improve each stage afterwards, instead of perfecting stages one at a time.

---

11. Team Roles

Adjust to team size. For four members:

Role	Responsibilities
Member 1: Data and evaluation lead	Dataset prep, gold set annotation coordination, metrics, statistical analysis
Member 2: Retrieval lead	Wikipedia indexing, BM25 and dense retrievers, retrieval evaluation

Role	Responsibilities
Member 3: Verification lead	Claim decomposition and verifier implementation, fine-tuning
Member 4: Repair and demo lead	Repair prompts, re-verification loop, demo app
Everyone	Annotation, error analysis, report writing, code review

---

12. Tech Stack and Tooling

- **Language:** Python 3.10+
- **Core libraries:** `transformers`, `sentence-transformers`, `faiss-cpu` or `faiss-gpu`, `rank-bm25` (or Pyserini), `datasets`, `scikit-learn`, `pandas`, `numpy`
- **LLM access:** an API (OpenAI, Anthropic, or similar) or open models via HuggingFace or Ollama, depending on budget
- **NLI models:** DeBERTa-based models from HuggingFace
- **Experiment tracking:** Weights & Biases or a shared spreadsheet
- **Demo:** Gradio or Streamlit
- **Version control:** GitHub, with a branch per feature and pull-request review
- **Compute:** Google Colab or Kaggle (free GPU) or university cluster

Cost control: cache every LLM and search call to disk, develop on small subsets, and run the full test set only a few times.

---

13. Risks and Mitigations

Risk	Impact	Mitigation
Evidence missing from Wikipedia	True claims flagged as Unverifiable	Treat Unverifiable as its own class and report it separately; optionally add web search
Poor claim decomposition	Errors propagate through the entire pipeline	Validate on a manual sample; iterate prompts early

Risk	Impact	Mitigation
LLM-as-judge is noisy or biased	Unreliable detection	Calibrate against human labels; compare with NLI and the fine-tuned verifier
Repair over-edits or introduces new errors	Lower preservation and new hallucinations	Strict prompts, localized edits, re-verification loop, over-edit metric
Data leakage (models may have seen benchmarks)	Inflated results	Keep a held-out test set, build our own gold set, acknowledge this in the report
API cost or rate limits	Delays	Caching, small subsets, open-source fallback models
Annotation takes longer than planned	Weak repair evaluation	Start annotation in Week 2; limit the gold set to about 100 answers if needed
Scope creep	Unfinished project	Fix scope in Phase 0; treat stretch goals as optional

---

14. Expected Deliverables

1. **Source code repository** with a README and reproduction instructions.
2. **Processed datasets and our annotated gold set** (subject to the licenses of the source datasets).
3. **Final report** (problem, related work, method, experiments, analysis, limitations).
4. **Results:** detection and repair tables, ablations, error analysis.
5. **Interactive demo** that highlights unreliable claims and shows the repaired answer.
6. **Presentation slides** (and optionally a short recorded demo).

---

15. Limitations and Future Work

15.1 Known limitations (to state honestly in the report)

- Verification is only as good as the **retrieved evidence**; a claim missing from the corpus cannot be confirmed.
- The scope is **factual claims in English open-domain QA**. Reasoning errors, opinions, and long multi-step logic are out of scope.
- The gold set is **small and annotated by the team**, so it may not reflect real-world error distributions.
- LLM components make parts of the pipeline **non-deterministic**; we mitigate this with fixed seeds, temperature 0, and caching.
- Wikipedia may be outdated for recent events.

15.2 Possible extensions

- Claim-triplet extraction (subject, predicate, object) for more precise matching.
- A **rule-based or deterministic repair path** for simple cases (for example, direct entity or number substitution from evidence), compared against the LLM repair. This echoes the deterministic-versus-probabilistic argument in the code-hallucination paper.
- **Citation generation:** attach the supporting evidence to every claim in the repaired answer.
- Extension to **summarization** (faithfulness hallucinations) using FaithBench.
- **Multilingual** verification (for example, Poly-FEVER).
- Integration as a **post-processing layer** for a chatbot or RAG system.

---

16. References and Reference Repositories

Note for the team: verify each link and read each README before relying on it.

16.1 Papers

- Khati, Rodriguez-Cardenas, Pantzer, Poshyvanyk. *Detecting and Correcting Hallucinations in LLM-Generated Code via Deterministic AST Analysis.* FORGE '26 (arXiv:2601.19106). *(Design inspiration: decompose, verify against ground truth, edit locally, and evaluate detection and fix separately.)*
- Gao et al. *RARR: Researching and Revising What Language Models Say, Using Language Models.* 2022.
- Dhuliawala et al. *Chain-of-Verification Reduces Hallucination in Large Language Models.* 2023.

- Min et al. *FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation.* 2023.
- Manakul et al. *SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative LLMs.* 2023.
- Chern et al. *FacTool: Factuality Detection in Generative AI.* 2023.
- Hu et al. *RefChecker: Reference-based Fine-grained Hallucination Checker and Benchmark for LLMs.* 2024 (arXiv:2405.14486).
- Mishra et al. *Fine-grained Hallucination Detection and Editing for Language Models (FAVA).* 2024.
- Li et al. *HaluEval: A Large-Scale Hallucination Evaluation Benchmark for LLMs.* EMNLP 2023 (arXiv:2305.11747).
- Lin et al. *TruthfulQA: Measuring How Models Mimic Human Falsehoods.* 2021.
- Thorne et al. *FEVER: a Large-scale Dataset for Fact Extraction and VERification.* 2018.
- Wang et al. *Factcheck-Bench: Fine-Grained Evaluation Benchmark for Automatic Fact-checkers.* 2023 (arXiv:2311.09000).
- Bao et al. *FaithBench: A Diverse Hallucination Benchmark for Summarization by Modern LLMs.* 2024 (arXiv:2410.13210).
- Huang et al. *A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions.* 2023 (arXiv:2311.05232).

16.2 Reference repositories

Project	**Repository**
RefChecker	https://github.com/amazon-science/RefChecker
FActScore	https://github.com/shmsw25/FActScore
SelfCheckGPT	https://github.com/potsawee/selfcheckgpt
FacTool	https://github.com/GAIR-NLP/factool
HaluEval	https://github.com/RUCAIBox/HaluEval
FELM	https://github.com/hkust-nlp/felm
FAVA	https://github.com/abhika-m/FAVA
VeriScore	https://github.com/Yixiao-Song/VeriScore
LongFact / SAFE	https://github.com/google-deepmind/long-form-factuality

Project	Repository
RARR	`google-research/language` repository, `rarr` folder
Vectara HHEM	HuggingFace: `vectara/hallucination-evaluation-model`
Hallucinations Leaderboard	https://huggingface.co/spaces/hallucinations-leaderboard/leaderboard
Code-hallucination paper replication package	https://github.com/WM-SEMERU/Hallucinations-in-Code

---

End of document.